

Three ways to search binary embeddings

A guide to the algorithms, their costs, and the evidence

11 September 2026

1 Starting point

I started this project after reading Exa’s description of its vector database [1]. If we look at the work inside a selected cluster, one thing we can explore is whether the query can help us choose documents before we score them. We can always calculate a score for every document and return the best ones, but that gets expensive as the collection grows. Skipping documents might help, so long as the work of choosing them costs less than the scores we avoid, and we still find the results we wanted.

We’ll compare three ways to search the same binary document embeddings. In method A, we open selected clusters and scan their documents. In B, we open clusters and then split the documents using bitplanes. In C, we open clusters and look up bit patterns, starting with the pattern the query prefers. We’ll call C *weighted key lookup*; this is the concrete version of starting at the ideal answer and “walking backwards.”

If we fix the document count, RAM budget, and required recall, which method gives us the lowest latency? Recall tells us how many of the correct reference results we recover, so we need to keep it in the comparison as we change the search. And if we want to estimate latency, counting calculations is only part of the job: we also need to look at the memory those calculations read. Avoiding a document score can introduce work elsewhere.

The experiments give us three different outcomes to explain. On the tested real embeddings, we did not find reliable evidence that either alternative was faster than a well-tuned scan. With constructed queries whose important coordinates stayed in one prefix, key lookup had a small advantage over a closely related directly routed scan. When the important coordinates changed with each query, bitplane branching had a large advantage in the constructed case. So the useful question is what makes those cases different, rather than which method wins everywhere.

Let’s start with one query and six documents, follow it through each method, and then count the work and storage. From there, we can look at recall, choose parameters, and compare the measurements. At the end we’ll use the calculations to consider a billion documents. Those larger latency numbers are forecasts; the complete C++ indexes were tested through eight million documents.

1.1 A query example

We use the binary document representation and floating-point query described in Exa’s post [1]. In our examples, a stored bit of 1 means +1, and a bit of 0 means −1. We’ll use five coordinates so we can follow the calculation on the page. Let the query be

$$q = [0.7, 0.5, -0.4, -0.6, 0.3].$$

With that same dot-product score, signs 11001 make every contribution positive, giving $0.7 + 0.5 + 0.4 + 0.6 + 0.3 = 2.5$. This is the pattern the query would prefer. Next we’ll

compare it with the documents that actually exist.

Document ID	Stored signs	First-bit cluster	Full score
0	01001	0	1.1
1	00100	0	-1.3
2	10100	1	0.1
3	11001	1	2.5
4	11000	1	1.9
5	00011	0	-1.1

Here the correct top two are documents 3 and 4. If scores tie, we put the smaller document ID first, and then use that same rule when checking every method. We can also run this example against the C++ implementations with `examples/small_search.py`.

If we look at coordinate 1, a matching sign contributes +0.7, while a mismatch contributes -0.7. That one change lowers the score by 1.4. At coordinate 5, the loss is only 0.6, so the query magnitude tells us how expensive a wrong sign would be. We can use that observation to guide the search. For a query with d coordinates, write its ideal score as $Q = \sum_{i=1}^d |q_i|$. Writing b_i for the decoded sign (+1 or -1), if the mismatching coordinates have total magnitude p , then

$$S(q, b) = Q - 2p, \quad p = \sum_{i: b_i \neq \text{sign}(q_i)} |q_i|. \quad (1)$$

For document 4, only coordinate 5 mismatches, giving $S = 2.5 - 2(0.3) = 1.9$. From here, ranking by the largest score or the smallest mismatch penalty gives the same order. Ordinary Hamming distance counts each mismatch equally; here we use the query magnitudes as weights.

1.2 Shared pipeline

Suppose the collection contains N documents. Before any queries arrive, we embed the text, choose a representation width d , pack the document signs, and build the index. We also assign each document to one cluster. A cluster is a stored subset of the collection, and we can form those subsets by learning from the vectors or by using a fixed sign prefix.

For shortening embeddings, we also refer to Exa’s use of Matryoshka training [1, 2]. We’ll use that representation as given. One distinction matters for the proposed search: a useful prefix is not the same thing as the largest query coordinates. Matryoshka doesn’t sort each query by magnitude, and it doesn’t guarantee that its first 24 sign bits preserve its nearest neighbors. Shortening the vector and taking its signs are two separate changes; we hold the resulting representation fixed across the methods we compare.

Once a query arrives, we embed and normalize it, select clusters, and search their documents. Opening one cluster is one *probe*. With C clusters and P probes, balanced clusters would give us about NP/C selected documents. If the clusters aren’t balanced, we need to add their actual sizes instead. And if we want to search the whole collection, we can set $C = P = 1$.

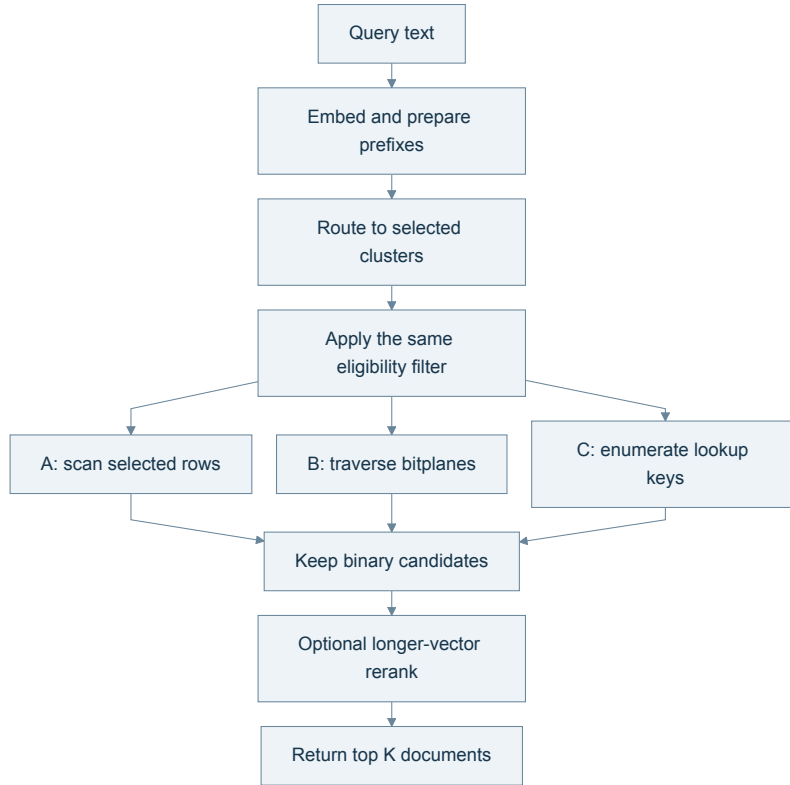


Figure 1: The common query flow. The three local search methods share the same query and selected clusters when we isolate their work. Longer-vector reranking is optional; it is excluded from the primary binary-score comparison.

A production pipeline can also filter documents or rerank candidates with longer vectors or another model. If those stages are enabled, they belong in the complete request measurement. For the primary index comparison here, we use cached query embeddings, the same binary score, and no float reranker. Later we’ll measure actual text queries so we can see the complete wait.

The baseline therefore follows Exa’s public description of compact scoring inside selected clusters [1]. The measurements here are of our local implementation, not Exa’s production system. We also try two ways of forming clusters: learning groups from similar vectors, known as IVF, and assigning documents by their first few sign bits. With a k -bit prefix, there are up to 2^k possible patterns, not k clusters. B and C change how we search the selected documents; they do not introduce the shared embedding or quantization techniques.

2 Search methods

2.1 Full scan

Let’s first scan the example so we have a result we can check. Coordinate 1 defines our two clusters; opening cluster 1 and then cluster 0 gives us documents 2, 3, 4, followed by 0, 1, 5. We calculate each full score and keep the best two. If we visit the documents in another order, the final ranking is still the same.

We keep a list of the best K documents found so far. In the implementation this list is a *heap*, which makes the worst retained result quick to find and replace. For $K = 2$, documents 2 and 3 give scores 0.1 and 2.5. Document 4 then scores 1.9, so it replaces document 2. None of the remaining documents improves the pair. This is all the result-selection code needs to do: keep the current best results as new scores arrive.

Listing 1: Full scan. Every selected document receives a full score.

```
scan(query, selected_clusters, K):
    table = prepare_score_table(query)
    best = empty_top_k(K)
    for cluster in selected_clusters:
        for document in cluster.documents:
            score = full_score(table, document.code)
            best.offer(score, document.id)
    return best.sorted_by_score_then_id()
```

For `full_score`, we use the lookup-table idea described in Exa’s dot-product optimization [1]. Each group of four binary document coordinates has 16 possible sign patterns, so we calculate their contributions once per query and reuse them. A 256-bit document then needs 64 table terms. We include this detail because it determines the baseline’s cost: a term still needs bit extraction, a lookup, and an addition. It is not one CPU instruction.

With g coordinates per group, the same idea gives $G = \lceil d/g \rceil$ groups and $G2^g$ stored table values, allowing padding in the last group. We prepare the table for each query, and all three methods use it. If we increase g , we trade a larger table for fewer terms per document. The measured implementation uses $g = 4$.

Let $\mathcal{P}$ be the selected clusters and n_c the size of cluster c . A scores

$$F_A = \sum_{c \in \mathcal{P}} n_c$$

documents and evaluates GF_A table terms. A useful local time model is

$$T_A^{\text{local}} \approx T_{\text{table}} + F_A c_{\text{scan}} + T_{\text{select}}(F_A, K). \quad (2)$$

This equation adds three pieces of time: prepare the score table, score the selected documents, and keep the best results. The value c_{scan} is the measured time to read and score one document when the rows are next to each other in memory. If that measured value already includes result selection, we must not add selection again. The heap has a worst-case cost of $O(F_A \log K)$, although many scores are simply rejected without changing the heap.

2.2 Scan memory

Now let’s look at what we store. If a packed word has w bits, it takes $u = w/8$ bytes, and a code needs $u \lceil d/w \rceil$ bytes. We also need the document ID; call its width b_{id} . Adding those two parts for every document gives

$$M_{\text{rows}} = N (u \lceil d/w \rceil + b_{\text{id}}). \quad (3)$$

For $d = 256$, $w = 64$, and eight-byte IDs, that is $32 + 8 = 40$ bytes per row, or 40 MB for one million rows in decimal units. We still need to account for cluster directories, spare

vector capacity, and the memory manager’s own records; the row data does not include them.

For example, six eight-byte IDs contain 48 bytes of data. If their array reserves space for eight IDs, the array uses 64 bytes even while only six entries are filled. That is the difference between data size (often called payload) and allocated memory. Total process RAM also includes the rest of the running program.

Choosing clusters takes memory and time too. A learned cluster has a representative vector, called its *centroid*, which the query can be compared with. If we store C float centroids of width d_r , their coordinates take $4Cd_r$ bytes. Comparing the query with all of them examines Cd_r coordinates, and then we choose the best P clusters. A router based on sign bits does different work, so its cost must be measured separately.

During a query, we also store the query vector, score table, selected cluster IDs, and current results. Call this temporary memory Q_0 ; its main terms grow as $O(d + G2^g + P + K)$. If Python and C++ result arrays exist at the same time, count both. Adding Q_0 to the stored index and its supporting structures gives the memory needed while serving a query. Building the index can need more because the input arrays and new index can exist together.

The scan reads consecutive document rows. This lets the CPU prepare later memory reads while working on the current ones; SIMD instructions can also process several values together. These are the kinds of hardware optimizations discussed in Exa’s post [1]. They do not change the number of documents scored, but they can change how long each score takes. That is why a method with fewer arithmetic operations is not automatically faster.

2.3 Bitplanes

The query tells us which sign it prefers at each coordinate. One thing we can try is selecting documents through that sign before calculating their full scores. For this, B stores a *bitplane*: one coordinate across all documents in a cluster, with the sign data arranged as columns. We also keep the packed document rows, because selected candidates still need the same full score as A.

Consider cluster 1 in the example, where the local document order is [2, 3, 4] and the codes are 10100, 11001, and 11000. If we read coordinate 2 down those rows, we get plane 011: documents 3 and 4 have that bit set. Coordinate 3 gives 100. The query prefers 1 at coordinate 2 and 0 at coordinate 3, so either selection keeps documents 3 and 4.

We can record the remaining local positions in a candidate mask. Starting with parent mask A and plane B_i , a positive-query split keeps the matching positions in one child and the others in another:

$$A_{\text{match}} = A \& B_i, \quad A_{\text{other}} = A \& \neg B_i.$$

If the query coordinate is negative, we swap which child is the match. Both results are intersected with the parent, so padding positions can’t become real documents.

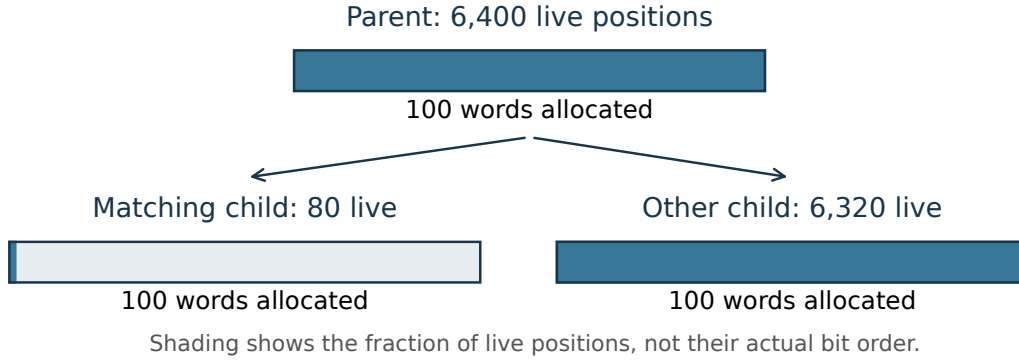


Figure 2: Membership shrinks, but a dense mask keeps its physical width. Each child still has 6,400 positions, or 100 words. This larger illustration counts storage; the six-document example explains the returned IDs.

2.4 Branch search

If we order the query coordinates by magnitude, we get 1, 4, 2, 3, 5. In cluster 1, coordinate 1 is already fixed, and all three documents match the query at coordinate 4. The split on coordinate 2 then gives [3, 4] in one child and [2] in the other. Matching adds no penalty, while the other child adds 0.5, so we try the matching child first.

We still need to keep the other child. Matching one bit does not rule out several bad matches later, so the first child we visit may not contain the best document. The search keeps the waiting branches in a queue: a list from which it chooses the most promising branch next. Each branch records the mismatch cost already known from its earlier splits.

Equation 1 lets us calculate the best score a branch could possibly contain. If its known penalty is p , that score is at most $Q - 2p$, even if every remaining coordinate matches. This maximum is called an *upper bound*. If it is strictly below the worst score in our current top- K results, we can skip the branch. For example, with ideal score 2.5 and known penalty 0.5, the branch cannot score above 1.5. If our current second-best score is 1.9, that branch cannot replace it. Equal scores still need the ID tie rule; the C++ check also allows for small floating-point rounding differences.

Listing 2: Bitplane search. The queue preserves alternatives to the preferred path.

```
branch(query, clusters, K, leaf_size, node_budget):
    table = prepare_score_table(query)
    order = coordinates_by_decreasing_abs(query)
    queue = one full candidate mask per selected cluster
    best = empty_top_k(K)
    while queue is not empty:
        node = remove_lowest_penalty(queue)
        if cannot_improve(node, query, best): continue
        if node_budget_is_reached(): return best
        if node.count <= leaf_size or no_bits_remain(node):
            score_each_set_document(table, node.mask, best)
        else:
            matching, other = split_on_next_bit(node, order)
            queue_nonempty_children(queue, matching, other)
    return best.sorted_by_score_then_id()
```

In the listing, a *node* is one group of remaining candidates. When a group is small enough, we stop splitting and score its documents; this final group is called a *leaf*. The leaf size L sets that threshold. For example, with a threshold of two, a group containing only documents 3 and 4 can be scored immediately. The C++ source also records work counts, manages memory, and handles small rounding differences.

A node budget limits how many groups we visit. It can stop the search before we have ruled out every better result, so it may lose recall. With no work limit and safe score bounds, we can finish exactly within the selected clusters. This still says nothing about a document whose cluster was never opened.

The exploration probability gives us another setting to try. Sometimes the search chooses a different waiting branch instead of the one with the best possible score. The score itself does not change, only the visit order. With a fixed amount of work this can help or hurt, so we repeat the test with different random seeds. The real-data tests did not show a consistent benefit from this random exploration.

2.5 Branch costs

For cluster c , each dense mask spans $W_c = \lceil n_c/w \rceil$ words. Let J_c count its splits and E_c the leaf masks whose set bits are enumerated. The full-width visits are

$$V_{\text{split}} = \sum_{c \in \mathcal{P}} J_c W_c, \quad V_{\text{leaf}} = \sum_{c \in \mathcal{P}} E_c W_c. \quad (4)$$

Even at a leaf, the loop checks zero words to find out that they are empty, and then the set bits tell us which documents to score. If we count only the surviving documents, we miss that first part of the work. An early cost estimate was too optimistic for exactly this reason: it omitted the leaf-word visits.

Let's follow the larger mask example. We start with 6,400 documents and narrow the candidates to 80, but the mask still contains 100 words. If we make ten more dense splits, those loops visit 1,000 words even if only a few bits remain set. Each visit reads the parent and plane, constructs children, and writes results; we also spend time allocating masks and maintaining the queue. These costs remain even when the candidate count looks small.

If F_B documents are eventually scored, we add the following time costs:

$$T_B^{\text{local}} \approx T_{\text{table}} + T_{\text{order}} + V_{\text{split}}c_{\text{split}} + V_{\text{leaf}}c_{\text{leaf}} + F_Bc_{\text{gather}} + T_{\text{queue}} + T_{\text{select}}(F_B, K). \quad (5)$$

To use this equation, we need to know what each measured price includes. The split price should cover the chosen mask allocation policy, and scattered row reads may cost more than contiguous scan reads. If a leaf timer already covers finding set bits, fetching the rows, and scoring them, we can use that combined measurement. Adding those same stages separately would count them twice.

The additional memory for B's stored bitplanes is

$$M_{\text{planes}} = du \sum_{c=1}^C \lceil n_c/w \rceil. \quad (6)$$

For one large cluster, these extra stored bits take approximately $Nd/8$ bytes. Many small clusters need additional padding because each cluster uses whole words. During a query we also keep masks for waiting branches. If $A_c(t)$ masks for cluster c exist at time t , the largest amount of mask data held at once is $u \max_t \sum_c A_c(t)W_c$. In words: count the live masks, multiply by their full widths, and take the largest total during the search. Then add the memory used to track those branches and the shared temporary data. Counting only the mask currently being searched would miss the waiting masks.

2.6 Key lookup

C uses a fixed set of h coordinates as a short bit pattern, or *key*. Within each cluster, we store documents with the same key together. A lookup table, called the directory in the code, tells us where that group starts and how many documents it contains. These matching documents are sometimes called a posting list; here we will simply refer to the documents for a key. Their existing document IDs are enough, so we do not add another ID array.

Let's use coordinates 2 and 3 as the key in the same example. The query wants 10. If we flip coordinate 3, the penalty is 0.4; flipping coordinate 2 costs 0.5, and flipping both costs 0.9. That gives us the weighted order 10, 11, 00, 01 to try in the directory.

Query key: coordinates 2 and 3 = 10

Key order	Known penalty	Cluster 0 IDs	Cluster 1 IDs
10	0.0	0	3, 4
11	0.4	empty	empty
00	0.5	5	empty
01	0.9	1	2

Figure 3: One shared key sequence is looked up in both selected clusters. The ideal key already finds the top two, but that fact alone is not a safe stopping rule. Empty lookups still cost work.

The first key returns document 0 from cluster 0 and documents 3 and 4 from cluster 1. We score them fully and get 1.1, 2.5, and 1.9, leaving [3, 4] as the best pair. Can the next key improve it? Its known penalty is 0.4, so even its full-score upper bound is only $2.5 - 2(0.4) = 1.7$, below 1.9. Here that bound lets us stop before the empty 11 lookups. Figure 3 includes the wider sequence to show the directory and its empty keys; a walk without the bound or a work limit would inspect them too.

It would be tempting to stop as soon as we find K documents, but the short key hasn't checked the other coordinates. A matching document might score poorly on them, while another key contains a better full score. We therefore score the full code and keep an upper bound for unvisited keys, rather than treating the candidate count as proof that we've finished.

Listing 3: Weighted key lookup. Generate the key sequence once; charge every cluster lookup.

```
keys(query, clusters, K, candidate_target, lookup_limit):
    table = prepare_score_table(query)
    ideal = signs_of_fixed_key_coordinates(query)
    queue = [empty_flip_set_with_zero_penalty]
    best = empty_top_k(K)
    while queue is not empty:
        if remaining_bound_cannot_improve(queue, query, best): break
        state = remove_lowest_penalty(queue)
        key = ideal XOR state.flip_mask
        for cluster in clusters:
            if lookup_limit_is_reached(): return best
            count_one_lookup_including_misses()
            for document in cluster.directory.get(key, []):
                best.offer(full_score(table, document.code), document.id)
        if candidate_target_is_reached(): return best
        add_next_unique_flip_sets(queue, state)
    return best.sorted_by_score_then_id()
```

The search tries keys in increasing total mismatch cost. This is not just a count of flipped bits: two cheap flips can cost less than one expensive flip. The C++ code generates each combination once. For an unvisited key, $Q - 2p_{\text{key}}$ gives its best possible full score by assuming that all coordinates outside the key match. This is safe, but it may overestimate what the stored documents can actually achieve, causing extra searching.

We can limit the work with a positive candidate target or lookup limit; zero disables a limit in the C++ API. Either limit can stop early and lose recall. The target is checked after a key has been processed across all selected clusters, while the lookup limit can stop during that loop. To finish exactly within the selected clusters, we must instead exhaust potentially improving keys or prove that none remain, with score ties handled consistently.

2.7 Key-lookup costs

Let H count actual directory lookups and F_C the full scores. With E generated keys and no limit cutting a cluster loop, $H = PE$. An empty key contributes to H but not F_C . A

time model for the implemented policy is

$$T_C^{\text{local}} \approx T_{\text{table}} + T_{\text{key order}} + T_{\text{key queue}} + H_{C_{\text{lookup}}} + F_{C_{\text{posting}}} + T_{\text{select}}(F_C, K). \quad (7)$$

The documents for one key are stored next to each other. Reading many small groups may still cost more per row than reading one long group. There is also work to manage the waiting keys. The implementation generates one sequence of keys and uses it across clusters, so we count that work once, not P times.

Let U_c be the number of keys that have documents in cluster c . Each directory entry stores the key, a starting position, and a count. Those fields occupy $\sum_c U_c(b_k + 2b_o)$ bytes: four bytes for the key and eight for each range field in this build. The fields therefore add up to 20 bytes per entry, but that is not the full memory allocation.

The C++ hash table stores entries in separate memory blocks, reached through an array of pointers. This implementation is known as *separate chaining*. If there are B_c pointer slots, it needs $\sum_c (U_c b_{\text{node}} + B_c b_{\text{ptr}})$ bytes for the entry blocks and pointer array, plus the container and memory manager’s own records. The formula counts the actual stored pieces. We cannot use the 20-byte field total as though it included all of them.

If we widen the key, the groups of matching documents may get smaller, but useful keys may also take more work to reach. A very short key has the opposite problem: it finds documents easily while leaving nearly all the scoring work. So finding a nonempty key isn’t the same as finding the best documents. And if we want to relax a pattern to something like $1*0$, we need a different operation; an exact-key hash table doesn’t answer that wildcard request in one lookup.

3 Recall and settings

3.1 Measuring recall

We have ways to skip some work. To check whether they miss good results, we first score every document and save the correct top results. These are the *reference results*. Let $G_K(q)$ be that list of top- K IDs and $A_K(q)$ the IDs returned by the method being tested. For $K \leq N$, recall for query q is the fraction of reference IDs we recover:

$$R_K(q) = \frac{|G_K(q) \cap A_K(q)|}{K}. \quad (8)$$

The intersection symbol means IDs present in both lists. In our example, $G_2 = [3, 4]$. Returning $[3, 0]$ recovers one of the two, giving 50% recall. Returning $[3, 4]$ gives 100%. Finding many candidates is not enough: the correct IDs must be among them and remain in the final result list.

Here, “correct” means the ranking from the binary-document score in Equation 1. Comparing instead with full float vectors would also count changes caused by quantization. Human relevance labels ask whether a document’s text is useful for the query. These are different tests: matching the binary ranking does not by itself prove that the text is relevant or that a language model will answer correctly.

If we follow a reference document through the pipeline, we can lose it when its cluster isn't opened, or when local search misses it inside an opened cluster. Fix a query and choose one of its K reference IDs uniformly. Let V mean that we visit its cluster, and L mean that the local method returns it. Then

$$R_K(q) = \Pr(V \mid q) \Pr(L \mid V, q). \quad (9)$$

The first probability asks how often a reference document's cluster is opened. The second asks how often it is returned *given that its cluster was opened*. For example, suppose 8 of the correct 10 documents reach the local search and it returns 6 of those 8. Overall recall is $(8/10)(6/8) = 6/10$. We do not need to assume those two stages are independent.

The scan returns every reference result that reaches it, so its second probability is one. B and C can also miss documents inside an opened cluster if we stop them early. Across many queries, average the complete per-query recall; multiplying two separately averaged stage probabilities generally gives a different answer.

3.2 A recall model

We can calculate an average recall if we first specify how the example documents are generated. Suppose each document bit is chosen independently, with equal chance of 0 or 1. Now give m query coordinates a large magnitude a , and the remaining $d - m$ coordinates a small magnitude ϵ . We choose them so that

$$a > (d - m)\epsilon. \quad (10)$$

The left side is the weight of one important coordinate; the right side is the total weight of all the small coordinates. This condition makes one important mismatch more costly than all the small mismatches together. Therefore a document matching every important coordinate beats any document that misses one. This is an assumption we choose for the example, not a property promised by Matryoshka training.

Each document has probability $p = 2^{-m}$ of matching all m important signs. Across N independently generated documents, let Z be the number that match. This count follows a *binomial distribution*: a formula for how many successes occur when we repeat the same independent chance of success. It gives

$$\Pr(Z = z) = \binom{N}{z} p^z (1 - p)^{N-z}, \quad \mathbb{E}[Z] = Np.$$

Here $\Pr(Z = z)$ is the chance of exactly z matching documents. The symbol $\binom{N}{z}$ counts the ways those z documents can be chosen. The expression $\mathbb{E}[Z]$ means the average count across repeated random collections; it is not a guaranteed count for one collection.

If we open only the matching group and score all its rows, we find the whole top- K when $Z \geq K$. When $Z < K$, we find only Z of them. Recall is therefore $\min(Z, K)/K$. Multiply each possible recall by its probability and add:

$$\mathbb{E}[R_K] = \frac{1}{K} \sum_{z=0}^N \min(z, K) \binom{N}{z} p^z (1 - p)^{N-z} = \frac{1}{K} \sum_{i=1}^K \Pr(Z \geq i). \quad (11)$$

The second form is the same average written another way: add the chances of finding at least one match, at least two, and so on through K . It avoids adding a term for every possible number of documents. The example below shows the calculation with just two documents.

Consider $N = 2$, $m = 1$, and $K = 1$. The ideal group is empty with probability $1/4$, contains one document with probability $1/2$, and contains two with probability $1/4$. We can calculate expected recall directly: $0(1/4) + 1(1/2) + 1(1/4) = 3/4$. Notice that the average group size is one, but some groups are empty. If we substituted that average as though it were a guaranteed count, we would incorrectly predict 100% recall.

Ideal-group size Z	Probability	Recall@1
0 matches	$1/4$	0
1 match	$1/2$	1
2 matches	$1/4$	1

Figure 4: Two independent documents and one required matching bit. Opening only the matching group succeeds with probability $3/4$. The empty-group outcome explains why the mean group size is insufficient.

At $N = 10^6$ and $m = 13$, the ideal group averages about 122 rows. With $K = 100$, some groups can still be too small, so a complete B or C search may need to continue into other groups. Equation 11 describes stopping after the ideal group; it doesn't describe every search policy. If we increase a work budget, we have to consider which additional groups we visit and the scores and sizes of those groups.

For real queries, keeping half the coordinates does not mean keeping half the information needed to rank documents. Some bits may tend to change together, query weights differ, and cluster selection may miss good documents. All of these affect recall. An earlier estimate used a normal distribution—a probability model centered on an average—to approximate prefix and full scores. It was fitted to some queries and checked on others, but the Nomic check missed recall by as much as about 9.6 percentage points. That failed the proposed two-point limit, so we cannot use that estimate to claim real-data recall at a billion rows.

3.3 Choosing settings

Some choices change what we build: the code width d , cluster count C , routing method, and C's key coordinates and width h . Others can change from query to query: probes P , B's leaf size and node budget, C's lookup and candidate limits, and the requested K . If we change d , the reference ranking changes too, so we need to compare all methods again using that same representation.

We can use the cost equation to narrow the settings worth trying. Suppose B follows one promising path, each split halves the candidates, and the masks keep width $W = \lceil N/w \rceil$. After j splits, we expect $N2^{-j}$ rows to reach the scorer. If we temporarily leave out changes

in queue cost, this gives

$$T_B(j) \approx T_0 + jWc_{\text{split}} + Wc_{\text{leaf}} + N2^{-j}c_{\text{score}}. \quad (12)$$

If we make one more split, we save about $N2^{-(j+1)}$ scores and pay for another full-width split. It helps in this model when the saved scoring time exceeds Wc_{split} and the additional queue cost. For example, with $N = 6,400$ and $j = 5$, another balanced cut saves about 100 scores. We compare the time of those 100 scores with the time of a 100-word split; the two counts alone aren't comparable.

The number of splits must be a whole number, but we can first treat j as a continuous variable to find where this time formula stops decreasing. The derivative measures that change in estimated time. Setting it to zero gives a candidate depth:

$$\frac{dT_B}{dj} = Wc_{\text{split}} - N \ln(2)2^{-j}c_{\text{score}} = 0, \quad j^* = \log_2 \left(\frac{N \ln(2)c_{\text{score}}}{Wc_{\text{split}}} \right).$$

If the formula suggested 3.6 splits, for example, we would test the valid whole-number choices 3 and 4. We then check the score assumptions and required recall. The calculation assumes one useful balanced path; it doesn't cover arbitrary branch exploration. Also, since W grows with N , a larger collection doesn't automatically call for a larger optimal depth. The dense masks themselves still take more work as the collection grows.

The document counts also help us choose when to stop splitting. At one million rows, the 13-bit matching group contains about 122 rows on average. Its standard deviation is about 11; this measures how much the group count varies around the average, not a limit that every group must obey. A threshold of 128 can make a slightly larger group split again on an unimportant coordinate. The rough margin $122 + 3(11) \approx 155$ suggested trying $L = 160$. The preceding 12-bit group averages about 244 rows, so it will usually still split. We tested this setting; the rough margin alone does not prove it is best.

The complete choice for method a is

$$\min_{\theta \in \Theta_a} T_a(\theta) \quad \text{subject to} \quad R_a(\theta) \geq r, \quad M_a(\theta) \leq M_{\text{limit}}. \quad (13)$$

In this formula, θ is a set of settings, such as cluster count and leaf size. We want the smallest query time among settings that meet both the recall target and the RAM limit. A derivative can help with a known time formula, but N and RAM do not tell us how many documents a real cluster contains or how often routing misses a good result. We still need measurements. We compare the best *tested* choices, including good routing options for A and a large enough leaf for B to scan when further splitting is not useful.

4 Experiments

The formulas count the work. We then measure the complete search to check whether skipping scores actually makes the query faster. We compare real embeddings first, then use constructed inputs to isolate how much the query's strong coordinates matter. The saved source record identifies the runs behind these comparisons [5].

4.1 Experimental setup

A, B and C receive the same documents and floating-point queries, use the same C++ weighted binary scorer, and return the top 100 with score ties broken by document ID. When we measure recall here, we measure recovery of that exact binary ranking; these comparisons use neither a float reranker nor human relevance judgments.

The machine is an Apple M5 Max with 128 GiB of physical memory, while each worker runs under a separate 32 GB decimal memory budget using one CPU thread and one query at a time. A fresh process builds each configuration, runs it before timing to warm it, and then measures routing plus search. The outer timer includes the call from Python into C++, but cached-vector measurements exclude text encoding. Repeating these build-and-measure runs helps expose timing variation; the recorded power settings do not prove that CPU frequency remained fixed.

We tune parameters on one set of query IDs, keep the choices unchanged, and evaluate them on different IDs, allowing each method its own fastest tested setting that meets the recall requirement. For A, the tested options include intermediate probe counts and direct routing by sign keys. For B, they include large leaves, so it can score a selected cluster without unnecessary splits. If we left these options out, a poor baseline or a forced traversal could decide the result before the timing begins.

The real study uses two rules for selecting settings. The first accepts a setting when its average tuning recall reaches the target. The more cautious rule repeatedly draws samples from the tuning queries and requires a lower recall estimate to reach the target. This leaves more margin, but cannot promise recall on future queries. To establish a speedup, both methods must meet the target, and the full 95% interval for A’s time divided by B’s or C’s time must exceed one. A ratio above one means A took longer. We keep each query’s method timings paired when resampling queries and whole process blocks; repeats are not extra independent queries.

4.2 Real-data results

For the real study, the documents are 256-bit codes from Nomic embeddings of MS MARCO passages [4, 3]. After combining 6,821 tuning settings, the selected choices were evaluated on 200 query IDs excluded from the earlier 300-query collection. On this sample, all 36 cases chosen by the more cautious rule met their requested recall, compared with only 7 of 36 chosen by average tuning recall.

In the results from the more cautious rule at one million documents below, each row sets a common minimum recall requirement and shows each method’s median routing-plus-search time. If we compare methods within a row, their achieved recall also happens to be equal.

Target	A (ms)	B (ms)	C (ms)	Recall (%)
80%	0.675	0.699	0.751	82.53
90%	1.634	1.686	1.880	91.73
95%	3.360	3.430	3.706	95.47
99%	9.580	9.965	11.013	99.25

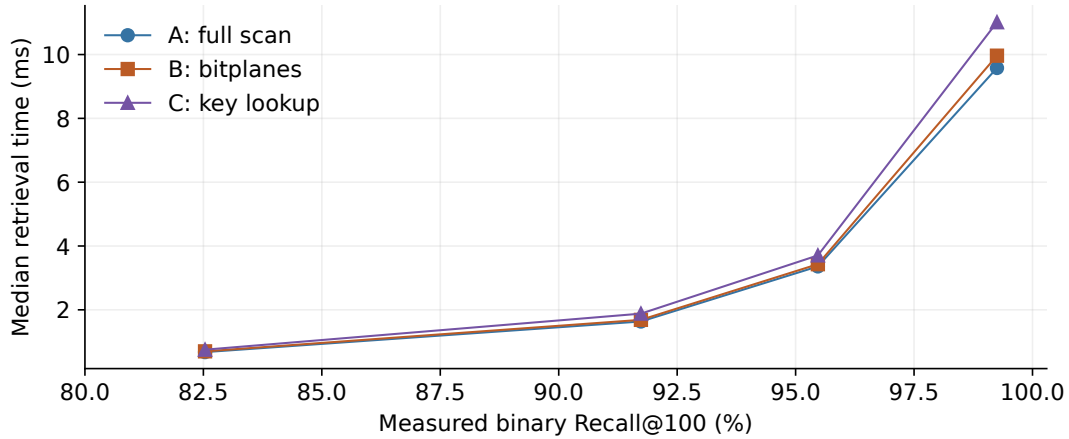


Figure 5: Four frozen recall-target choices on one million real passage embeddings and 200 independent query IDs. Lines connect the tested settings for readability; they do not predict intermediate settings. At each shown recall, A has the smallest median.

Across the real study, none of the 48 evaluated comparisons established a B or C speedup under the stated rule. Although fewer documents received full scores, that saving did not produce a supported reduction in complete retrieval time. This conclusion applies to this data, score, implementation and tested parameter range.

A later control gives C a one-bit key, A’s routing layout, and unlimited candidate and lookup budgets, while A and B keep their previously selected settings. This control uses 100 further query IDs, excluding all 500 previously used IDs. A, B and C measured 9.541, 9.855 and 10.545 ms, respectively, and all three achieved 98.95% recall, below the requested 99%. That remains a miss; we do not round it into a pass. The sample is separate from the earlier successful 200-query evaluation, and no parameter was changed in response to its outcome.

4.3 Controlled-data results

With constructed inputs, we can isolate the effect of keeping the strong coordinates fixed or changing them with each query. Document signs are independent and balanced across 256 coordinates. Each query has 13 strong coordinates of magnitude one, with magnitude 0.0001 on the remaining coordinates. A document matching every strong sign therefore beats any that mismatches one, regardless of the weak signs. The fixed case keeps the strong positions the same; in the changing case, each query chooses different strong positions uniformly without replacement. These vectors are constructed, not measured text embeddings.

The final follow-up uses 40 tuning queries and 20 separate evaluation queries, with three process blocks and three timing repeats. It adds neighboring key widths, cluster counts and leaf sizes, and then rereasures earlier choices. At one million documents and the 99% requirement, the results below show that all methods reach 100% recall except A in the changing case, at 99.95%.

Strong positions	A (ms)	B (ms)	C (ms)
Fixed	0.0140	0.0160	0.0128
Changing	24.9235	0.1446	14.6301

With fixed positions, A selects two of 16,384 clusters by direct routing, while C uses one physical cluster and a 13-bit key. The estimated ratio of A’s time to C’s is 1.098, with a paired 95% interval of 1.066–1.147, and the time difference is about 1.25 microseconds. Both methods select documents through the same strong signs, but C generates keys and scores their documents in one C++ call, whereas A routes and scans through separate calls. The small measured advantage therefore concerns this implementation of closely related candidate selection.

If the strong positions change, a fixed key cannot consistently select them. B can follow them because it orders coordinates using each query’s magnitudes; its selected setting uses one cluster, unlimited traversal and a leaf size of 160. A opens 255 of 256 centroid clusters, and the estimated ratio of A’s time to B’s is 172.3, with a paired 95% interval of 169.1–177.2. C uses a 14-bit key and has a lower median than A, but the paired interval for A’s time divided by C’s includes one, so that median alone does not establish an advantage.

We can see why this leaf size helps by calculating the number of documents in the group: at one million rows, the ideal 13-bit group has mean size about 122 and standard deviation about 11. A threshold of 128 can force another split on a weak coordinate, whereas a threshold of 160 leaves more room to score that group. The experiment checks this suggestion from the distribution; it does not establish that 160 is best for arbitrary embeddings, sizes or machines.

4.4 Checking the estimates

Before checking work or timing predictions, we need an independently calculated answer. The checks calculate scores directly from document signs and sort by score and ID; larger runs count strong and weak mismatches from packed bits to avoid a huge expanded matrix. Every complete C++ search call in the scaling check returned the independent reference ranking.

To check the predicted work, we compare it with counts recorded by the C++ search. Across the earlier controlled tuning and evaluation runs, 447 applicable per-query predictions matched exactly, including word visits during full-width splits and leaf scans, scored rows, attempted keys including empty lookups, and the highest tracked mask memory use where applicable. Queries outside the formulas’ stated conditions remain in the measurements, but do not count as successful predictions.

Even with correct counts, we cannot assign every row the same time cost. In a 16-million-document buffer, scoring about 1,958 scattered rows took 0.159 ms with reused candidates and 0.455 ms with changing candidates. Candidate generation stayed outside the timer, but still changed which memory had been accessed recently. Time models fitted to one- and four-million-row buffers underestimated a 16-million-row check by as much as 45.2% for reused candidates and 27.6% for changing candidates. Both failed models remain saved. The largest row store was about 640 MB, so these checks do not validate access costs for a 40 GB store.

The later model predicts the whole search call using one median time per collection size. The earlier fit gave fast individual queries too much weight when predicting a median. The revised model also includes the work of selecting results from C’s small lists of document IDs. For B, the leaf threshold grows as $L = 160N/10^6$, so a larger collection still stops splitting after the intended strong coordinates instead of forcing more cuts.

The revised four-million-row result had already been inspected, so it helps diagnose the model rather than independently test it. The model’s fitted values were fixed before the separate eight-million-row run below. A positive error means the predicted time was too high.

Positions	Method	Measured (ms)	Predicted (ms)	Error
Fixed	A	197.029	203.016	+3.0%
Fixed	B	0.854	0.884	+3.6%
Fixed	C	0.0481	0.0518	+7.7%
Changing	A	196.664	197.816	+0.6%
Changing	B	0.900	0.859	−4.5%
Changing	C	202.767	207.086	+2.1%

All six checks met the earlier 20% criterion: the model predicted times at a larger collection size under the same specified workload and existing query IDs. These model-check configurations are separate from the tuned comparisons in the preceding table. Although the largest observed error was 7.7%, that does not give us a guaranteed bound for future queries or a billion documents.

4.5 End-to-end timing

When a user supplies text, retrieval covers only part of the wait. The final check therefore runs 20 existing independent-query texts through the cached local Nomic model using Apple’s MPS GPU backend, normalizes each vector, routes it and calls C++ search, with a batch size of one. Three blocks and two repeats give 120 requests per method, or 360 requests in total. Each sequence shape is warmed, and the request timer excludes model loading, index building and reference creation.

Using the earlier unchanged settings, all three methods achieve 99.45% binary recall on this sample. The table gives the stage medians alongside the separately measured request total.

	Encode (ms)	Retrieve (ms)	Total (ms)	Total p95 (ms)
A	6.488	9.406	16.313	19.201
B	6.500	9.651	16.303	19.217
C	7.543	11.201	18.912	21.898

The median is the middle time; p95 is the time at or below which 95% of requests finish. The tiny difference between the A/B totals does not establish a speedup. If we add the stage medians, we need not get the total median, because the request with the middle

encoding time need not have the middle retrieval time. We must measure each request’s total before taking its median.

These timings apply to real Nomic text queries. Since the constructed strong-coordinate vectors do not come from this encoder, adding its time to their search measurements would not give us a measured text-to-results pipeline.

5 Larger collections

5.1 Total cost

To account for one complete request, let T_{shared} include query embedding, normalization, routing, and any common filtering. If T_a^{local} is the local cost for A, B, or C derived earlier, including its score table, we have

$$T_a^{\text{request}} = T_{\text{shared}} + T_a^{\text{local}} + T_{\text{final}}, \quad (14)$$

where T_{final} contains output work and any enabled reranking. All of these stages belong to the same request, so the equation does not justify adding unrelated stage medians or adding the real encoder’s measured time to constructed vectors it never produced.

For memory, we count the stored index separately from the temporary memory used during a query. Write M_0 for memory allocated to document rows, routing data, and common containers. Let D_C be C’s directory allocation, Q_B B’s extra masks and queue, and Q_C C’s extra key queue. All methods also use the common temporary query memory Q_0 .

Method	Stored index memory	Temporary query memory
A	M_0	Q_0
B	$M_0 + M_{\text{planes,allocated}}$	$Q_0 + Q_B$
C	$M_0 + D_C$	$Q_0 + Q_C$

The common data structures can reserve different amounts of unused space, so their actual allocations can differ. A process RAM limit must also count memory used to run the program and other data still in memory. We count each allocation once, even if both process and GPU reports include it. A limit during index building must also include temporary input and output copies.

5.2 Memory requirements

At $N = 10^9$, $d = 256$, and eight-byte IDs, packed codes occupy 32 GB and IDs occupy 8 GB, so these two arrays alone exceed a 32 GB budget. B also keeps a second sign representation, adding at least 32 GB before per-cluster padding. Changing how many clusters we open does not reduce these stored bytes.

Layout at 1B rows	Minimum data	Planned query RAM	64 GB budget
A, direct prefix route	40 GB	about 49 GB	Predicted to fit
C, global 13-bit key	40 GB	about 49 GB	Predicted to fit
B, global cluster	72 GB	about 83 GB	Rejected
B, prefix route, large leaf	72 GB	about 81 GB	Rejected

These estimates allow for reserved space and data structures, plus a configurable 1 GB reserve for running the program. That reserve is an assumption, not a memory measurement. In their current in-memory layouts, all three fail 32 GB. All listed planning cases fit 1 TB, but a machine with 1 TB need not have this machine’s memory performance. If we compressed the IDs or used another storage architecture, we would need to calculate memory again; relabeling the same index would not change the result.

5.3 Fixed bit positions

If we keep thirteen strong coordinates fixed under the earlier independent strong-coordinate model, the expected ideal group contains $10^9/2^{13} \approx 122,070$ rows, far more than the requested 100. The earlier probability calculation accounts for the small chance that fewer than 100 rows match, under these assumptions. It makes no claim about ordinary text embeddings.

A can route directly to this group and scan it, while C can look up its key and score the same rows. Under these assumptions, the model fitted to measured times predicts roughly 3 ms for both. B can also take this route with a large leaf, giving a similar time with more stored data. We should not force B through a global billion-row bitmap just to make C appear better. In this case, A and C are closely related ways to obtain the same candidate set, and the model cannot reliably distinguish a gap of a few percent.

5.4 Changing bit positions

If the important coordinates change with each query, a fixed prefix may contain few of the coordinates that determine the ranking, while B can choose its split coordinates from the query. We preserve the constructed path by keeping thirteen useful cuts and scaling the leaf size to 160,000. At the last split, the masks occupy approximately

$$(13 + 2) \lceil 10^9/64 \rceil \times 8 = 1.875 \text{ GB}.$$

The factor 15 counts masks on this path and queued alternatives held at the same time, including the parent while its children are created. This count applies to the specified path; other sequences of splits can need different amounts of temporary memory.

The model for the complete C++ search call predicts about 0.10 s for this global B path and about 25 s for a global scan. If we double B’s variable cost, we get about 0.20 s, which shows how the prediction changes when that assumed cost changes; it is not a confidence interval. No full billion-document query was measured.

We also need to give A the benefit of routing in this comparison. If it scores fraction f of the collection, its approximate cost is $T_{\text{route}} + fNc_{\text{scan}}$. Even with routing time ignored, A must score less than about 0.41% of the collection to beat the 0.10 s B prediction,

while still meeting the recall target at that fraction. Scanning can win if routing cheaply produces a small, accurate candidate set; if it cannot, the query-dependent B path is worth testing.

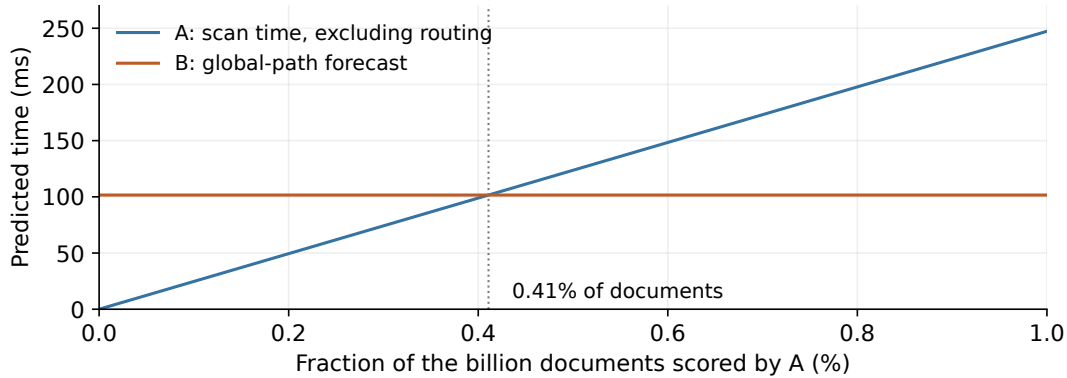


Figure 6: A conditional comparison with sufficient RAM. B is one prescribed operating point. A gets free routing in this line but must still attain the required recall at its scanned fraction. These are forecasts, not measured billion-document timings.

For C, the short key has to capture important coordinates. In the constructed changing case, a fixed 13-coordinate key misses all thirteen strong coordinates with probability about 49.94%. A lookup that finds documents can therefore still leave almost all the scoring work to do. The number of documents found by a key alone cannot establish faster search or better recall.

5.5 Conclusions

For the real embeddings measured here, the scan remains the supported practical baseline. The constructed comparisons show how that result depends on the workload: when the strong coordinates stay in the fixed prefix, direct routing and key lookup both avoid most scores; with changing strong coordinates, B’s query-dependent splits produced a large measured advantage. Each conclusion applies to its stated workload and tested parameter space.

We can use the formulas to explain these outcomes and suggest settings, but they do not prove that every possible IVF layout loses, that Matryoshka creates the constructed distribution, or that a fitted cost per operation remains exact at a billion rows. To test the favorable case on a real representation, the next useful experiment would check whether a few coordinates determine the ranking and whether routing finds the right documents, under the same recall and RAM limits.

6 Code and data

If we want to follow a calculation into the implementation, the table below gives the starting points in the companion repository. Paths are relative to its root. The full implementations, saved measurements, and longer derivations are there too: the detailed report remains in `reports/search-math/`, and this edition is in `reports/search-math-concise/`.

Question	Start here
Follow the six-document example	<code>examples/small_search.py</code>
Inspect scan and branching	<code>cpp/index.cpp</code> , <code>cpp/score.hpp</code>
Inspect key lookup	<code>cpp/key_index.cpp</code>
Check storage and work formulas	<code>experiments/models.py</code> , <code>experiments/check_controlled.py</code>
Read the final timing checks	<code>experiments/check_native_model.py</code> , <code>reports/search-math/evidence/final-study.json</code>
Calculate the larger cases	<code>experiments/project_costs.py</code> , <code>results/projected-cases-2026-09-11/</code>
Reproduce the study	<code>experiments/README.md</code> , <code>results/README.md</code>

Once the project is installed using its root README, we can run the six-document example without downloading a corpus. The second command checks the long report's embedded arithmetic and saved evidence. The third uses frozen inputs to recalculate the larger scenarios, so it doesn't build a billion-row index.

Listing 4: Three existing entry points, run from the repository root.

```
.venv/bin/python examples/small_search.py
python3 reports/search-math/verify_math.py
.venv/bin/python -m experiments.project_costs \
  --inputs results/projected-cases-2026-09-11/results.json \
  --output results/my-scenarios \
  --documents 1000000000 --ram-gb 32 64 1000
```

For the exact saved-input invocation and full-data runs, use the reproduction README. A full rerun also needs the dataset, embedding cache, and documented environment. Compiling this PDF only rebuilds the explanation: its tables summarize saved results, not new measurements.

References

- [1] Exa Team. *How we built a web-scale vector database*. 2024.
<https://exa.ai/blog/building-web-scale-vector-db>.
- [2] A. Kusupati et al. *Matryoshka Representation Learning*. 2022.
<https://arxiv.org/abs/2205.13147>.
- [3] Microsoft. *MS MARCO*. Passage collection and benchmark resources.
<https://microsoft.github.io/msmarco/>.
- [4] Nomic AI. *nomic-embed-text-v1.5*. Model documentation.
<https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>.
- [5] Local experimental record, 10–11 September 2026. Source checkpoint `f16ca66`; `reports/search-math/evidence/` and the result files it identifies. The concise edition's README records the selected sources and reproduction scope.